

CSCE 614 COMPUTER ARCHITECTURE · MODULE 11 · LECTURE NOTES

Multicore and Cache Coherence

Many caches, one memory, and the protocol that reconciles them.

MODULE OUTCOMES

- Explain why coherence is needed and what it guarantees.
- Describe the MESI protocol states and transitions.
- Explain false sharing and detect it.
- Explain why atomic operations cost what they do.
- Explain NUMA and its implications for data placement.

1 Why coherence is needed

Each core has private L1 and usually private L2 caches. They share main memory. Without a protocol, two cores can hold different values for the same address and neither will know.

Initially $x = 0$.

Core 0		Core 1
load x -> caches 0		load x -> caches 0
store $x=1$ -> its cache: 1		load x -> reads 0 from its own cache

Cache coherence is the guarantee that a read of a location returns the value of the most recent write to that location, regardless of which core performed it.

COHERENCE AND CONSISTENCY ARE DIFFERENT PROBLEMS

Coherence concerns a single memory location: all cores must agree on the sequence of values it took.

Consistency concerns the relative ordering of operations to *different* locations: if I write A then B, can another core observe B before A?

Coherence is handled entirely by hardware and is essentially invisible except as performance. Consistency is visible, is *not* what most programmers assume, and is Module 12. Conflating them is the source of a great deal of confusion about concurrent code.

2 MESI

State	Meaning	Other caches may hold it	Memory is
Modified	This cache has the only copy and has modified it.	No	Stale — this cache must write back on eviction.
Exclusive	This cache has the only copy, unmodified.	No	Current.
Shared	This cache has a read-only copy.	Yes	Current.
Invalid	This cache's copy is not usable.	—	—

- **Read miss.** Fetch the line. If no other cache holds it, enter **E**; otherwise all copies become **S**.
- **Write to a Shared line.** Broadcast an invalidate; all other copies become **I**; this one becomes **M**.
- **Write to an Exclusive line.** Transition silently to **M** — *no bus traffic at all*.
- **Another core reads a Modified line.** This cache supplies the data and drops to **S**; memory is updated.

WHY THE EXCLUSIVE STATE IS WORTH HAVING

Without E, a cache could not distinguish 'I am the only holder' from 'I share this'. Every write would have to broadcast an invalidate just in case.

With E, writes to genuinely private data — which is most data in most programs — generate no coherence traffic whatsoever. The state exists purely as an optimisation for the common case, and it is a large one.

Coherence is enforced by **snooping** (every cache watches a shared bus) in small systems, and by a **directory** (a central record of which caches hold which lines) in larger ones, since broadcasting does not scale past a handful of cores.

3 False sharing

THE DEFINING PROPERTY: COHERENCE TRACKS LINES, NOT VARIABLES

Two threads write to two entirely different variables. Those variables happen to lie within the same 64-byte cache line.

There is no logical sharing whatsoever. The hardware has no way to know that — its unit of coherence is the line. Every write by one thread invalidates the other thread's copy of the line.

The line now moves between cores on every single write, and each access becomes a coherence miss costing hundreds of cycles.

The result is a parallel program that can be **slower than its serial equivalent** while being perfectly correct and perfectly independent in the source. Nothing in the code suggests a problem, which is what makes this the most insidious bug in parallel programming.

```
// BAD -- all eight counters occupy one or two cache lines
long counters[8];
// thread i: counters[i]++;

// PADDED -- each counter gets its own line
struct alignas(64) Counter { long v; char pad[56]; };
Counter counters[8];

// BEST -- accumulate in a register, write once
long local = 0;
for (...) local++;
counters[tid].v = local;
```

The third form is generally preferable. Padding removes the false sharing but still writes to memory on every iteration; local accumulation keeps the hot variable in a register and touches shared memory exactly once. The general principle is to minimise writes to any line another core might touch.

Detection: a parallel speedup far below the core count with no obvious lock contention. Confirm with hardware counters for coherence misses (perf c2c on Linux is purpose-built for this and will name the offending cache line).

4 Atomics

An atomic read-modify-write must acquire the cache line in **Modified** state exclusively, perform the operation, and hold it until complete. Under contention, the line is pulled back and forth between cores continuously.

Situation	Typical cost
Uncontended atomic increment (line already M)	~20 cycles

Situation	Typical cost
Lightly contended (2–4 cores)	100–200 cycles
Heavily contended (many cores)	500+ cycles, worsening with core count

ATOMICS DO NOT SCALE

A single contended counter is a serialisation point. Adding cores makes it *worse*, because more cores compete for the same line and each transfer costs more as the interconnect grows.

This is one of the most common reasons a parallel program fails to scale while appearing to have ample parallelism.

The remedy is the same as for false sharing: give each thread its own counter, accumulate locally, and combine once at the end. The cost drops from $O(\text{operations})$ coherence transfers to $O(\text{threads})$.

5 NUMA

On a multi-socket system, each processor has memory attached directly to it. Accessing another socket's memory traverses an interconnect.

Access	Latency	Bandwidth
Local node	~90 ns	Full
Remote node	~140 ns	Reduced, and shared with coherence traffic

As Module 07 established, a physical page is allocated on the node of whichever core **first touches** it — not where `malloc` was called. The practical consequence is important and routinely missed:

INITIALISE IN PARALLEL

Allocating a large array and initialising it from a single thread places every page on that thread's node. Processing it afterwards from all threads means every other socket pays remote latency for every access, and the interconnect becomes a bottleneck.

Initialising the array *in parallel*, with each thread touching the region it will later process, distributes the pages correctly and can improve throughput substantially on multi-socket machines.

Pinning threads to cores (`numactl`, `sched_setaffinity`) prevents the scheduler from later migrating a thread away from its data, which would reintroduce the problem.

6 Writing code that scales

- 1 **Minimise sharing.** Per-thread state combined at the end beats shared state with locks, almost always.

- 2 **Pad concurrently written structures** to 64-byte boundaries, or use `alignas(64)`.
- 3 **Avoid contended atomics** in hot paths. Accumulate locally.
- 4 **Initialise in parallel** so NUMA first touch distributes pages correctly.
- 5 **Pin threads** when data placement matters.
- 6 **Measure coherence traffic** rather than guessing. `perf c2c` identifies the exact line and the exact source locations involved.

A parallel speedup substantially below the core count, with no visible lock contention, is almost always sharing of some kind — false sharing, atomic contention, or NUMA placement. Insufficient parallelism is a much rarer explanation than people assume.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Onur Mutlu — Cache Coherence, Multiprocessors lectures	MESI and directory protocols developed properly, with the scaling arguments.
Ulrich Drepper — What Every Programmer Should Know About Memory, section 3.3	Coherence and false sharing with measurements.
Paul McKenney — Is Parallel Programming Hard? (free book)	The practitioner's treatment of sharing, counters, and scalability. Chapter 5 on counting is directly relevant and unusually good.
perf c2c documentation	The tool that finds false sharing by cache line and source line.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write a parallel counter with threads incrementing adjacent array elements. Measure scaling from 1 to 8 threads and observe it failing. Then pad to 64 bytes and measure again.
- 2 Use `perf c2c` (or equivalent) to identify the false-sharing line in the unpadded version, and confirm it names the right source lines.
- 3 Measure the cost of an uncontended atomic increment, then with 2, 4, and 8 threads contending. Plot cost against thread count.
- 4 Implement the same counting workload three ways: a single atomic counter, padded per-thread counters, and local accumulation with a final combine. Measure all three at several thread counts.

- 5 Write a producer-consumer benchmark where producer and consumer write to adjacent variables, and show the effect of separating them onto different cache lines.
- 6 On a NUMA machine if available: allocate and initialise a large array serially, then process it in parallel; then initialise in parallel and repeat. Report the difference and verify page placement with numastat.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does cache coherence guarantee, and how does it differ from memory consistency?
- 2 Name the four MESI states. Why does the Exclusive state exist?
- 3 Explain false sharing and why it is invisible in source code.
- 4 Why does a contended atomic get *worse* as cores are added?
- 5 Give the single pattern that fixes both false sharing and atomic contention.
- 6 What is NUMA first touch, and what does it imply about how to initialise large arrays?